

Programação Funcional

Generalização em Haskell (Segunda Parte)

Vladimir Oliveira Di Iorio
DPI - UFV

Polimorfismo e Ordem Mais Alta

- Importantes funções que são polimórficas e de ordem mais alta, simultaneamente:
 - **map** : uma lista L2 é criada a partir de uma lista L1, aplicando uma operação sobre cada elemento de L1.
 - **fold** : um valor é calculado, resultado da aplicação de uma operação binária ao longo de toda uma lista de elementos.
 - **filter** : uma lista L2 é criada a partir de uma lista L1, selecionando alguns elementos de L1 que satisfazem a uma determinada propriedade.
- As funções `map`, `fold` e `filter` não impõem qualquer restrição sobre os tipos envolvidos.

Map

- Uma lista L2 é criada a partir de uma lista L1, aplicando uma operação sobre cada elemento de L1.
- Não há restrição sobre o tipo dos elementos de L1, que ainda pode ser diferente do tipo dos elementos de L2.

$$\begin{array}{ccccccc} L1 = [& e1 & , & e2 & , & \dots & , & ek &] \\ & \downarrow f & & \downarrow f & & & & \downarrow f & \\ L2 = [& f(e1) & , & f(e2) & , & \dots & , & f(ek) &] \end{array}$$

Map

```
1 times2, times3 :: Int -> Int
2 times2 n = 2*n
3 times3 n = 3*n
```

```
Main> map times2 [2,3,4]
[4,6,8]
Main> map times3 [2,3,4]
[6,9,12]
```

Map

```
1 isEven :: Int -> Bool
2 isEven n
3   | n `mod` 2 == 0    = True
4   | otherwise         = False
```

Map

```
1 isEven :: Int -> Bool
2 isEven n = (n `mod` 2 == 0)
```

```
Main> map isEven [2,3,4]
[True,False,True]
```

Map

```
1 offset = ord 'a' - ord 'A'
2 isCapital ch = (ch >= 'A') && (ch <= 'Z')
3 small :: Char -> Char
4 small ch
5     | isCapital ch      = chr (ord ch + offset)
6     | otherwise         = ch
```

```
Main> map small "ABC Esporte Clube"
"abc esporte clube"
```

Map

```
1 length [] = 0
2 length (a:x) = 1 + length x
```

```
Main> map length ["ABC","Esporte","Clube"]
[3,7,5]
Main> map length [2,3,4]
ERRO DE TIPO!!!
```

Map

- Definição:

```
1 map :: (t -> u) -> [t] -> [u]
2 map f [] = []
3 map f (a:x) = f a : map f x
```

- Alternativa:

```
1 map :: (t -> u) -> [t] -> [u]
2 map f l = [ f a | a <- l ]
```

Fold

- Um valor é calculado, resultado da aplicação de uma operação binária ao longo de toda uma lista de elementos.
- Não há restrição sobre o tipo dos elementos da lista.

`fold f [ e1 ] = e1`

`fold f [e1 , e2 , ... , ek]
= e1 `f` (e2 `f` (... `f` ek) ...)
= f e1 (fold f [e2 , ... , ek])`

Fold

```
Main> fold (+) [1..3]
6
Main> fold (||) [False,True,False]
True
Main> fold (++) ["ABC"," Esporte", " ", "Clube"]
"ABC Esporte Clube"
Main> fold (-) [1..3]
2
```

Fold

```
1 maxi :: Int -> Int -> Int
2 maxi a b
3   | a > b           = a
4   | otherwise       = b
```

```
Main> fold maxi [2,4,3]
4
Main> fold maxi [2]
2
Main> fold maxi []
ERRO!!!
```

Fold

- Definição:

```
1 fold :: (t -> t -> t) -> [t] -> t
2 fold f [a] = a
3 fold f (a:b:x) = f a (fold f (b:x))
```

- Mas os tipos da definição podem ser mais gerais...
- Qual a restrição sobre o tipo da operação f que será executada ao longo da lista?

Fold

- Supondo uma lista $[e_1, \dots, e_k]$ de tipo $[t]$:

$\text{fold } f \ [e_1, e_2, \dots, e_k]$
 $= e_1 \text{ `f` } (e_2 \text{ `f` } (\dots \text{ `f` } e_k) \dots)$
 $= f \ e_1 \ (\text{fold } f \ [e_2, \dots, e_k])$

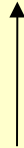
tipo do primeiro
argumento de f
tem que ser t

tipo do segundo
argumento de f tem que
ser o mesmo do valor de
retorno de fold

Fold

- Supondo uma lista $[e_1, \dots, e_k]$ de tipo $[t]$:

```
fold f [ e1 , e2 , e3 ]  
  = f e1 (fold f [ e2 , e3 ])  
  = f e1 (f e2 (fold f [ e3 ]))  
  = f e1 (f e2 e3)
```



tipo do valor de retorno de f deve ser o mesmo tipo do segundo argumento de f

Fold

- Restrições:

- tipo do primeiro argumento de **f** tem que ser **t**, o tipo dos elementos da lista.
- tipo do segundo argumento de **f** tem que ser o mesmo tipo do valor de retorno de **fold**.
- tipo do valor de retorno de **f** deve ser o mesmo tipo do segundo argumento de **f**.

```
1 fold :: (t -> u -> u) -> [t] -> u
2 fold f [a] = a
3 fold f (a:b:x) = f a (fold f (b:x))
```

Fold

- Acrescentando um argumento a fold:

```
1 fold1 :: (t -> u -> u) -> u -> [t] -> u
2 fold1 f s [] = s
3 fold1 f s (a:x) = f a (fold1 f s x)
4
5 sumListInt l = fold1 (+) 0 l
```

```
Main> sumListInt [1..3]
6
Main> sumListInt [3]
3
Main> sumListInt []
0
```

Filter

- Uma lista L2 é criada a partir de uma lista L1, selecionando alguns elementos de L1 que satisfazem a uma determinada propriedade.
- Não há restrição sobre o tipo dos elementos de L1, mas é o mesmo tipo dos elementos de L2.

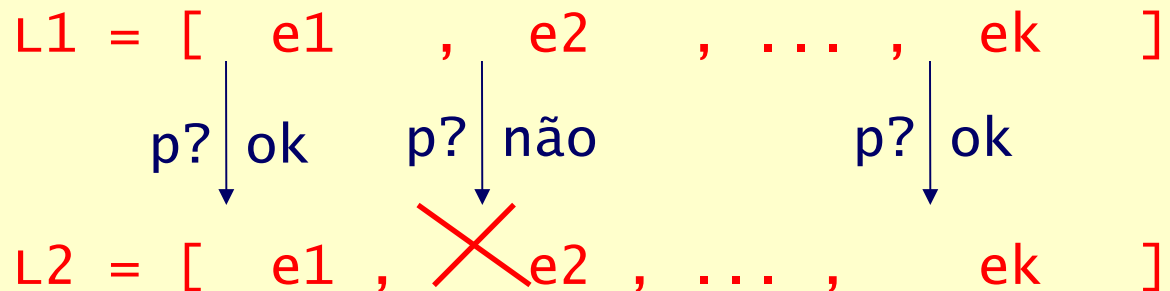
$$L1 = [e1, e2, \dots, ek]$$

$$\begin{array}{ccccc} & \downarrow p? \text{ ok} & \downarrow p? \text{ não} & & \downarrow p? \text{ ok} \\ L2 = [& e1 & , \text{X} e2 & , \dots , & ek] \end{array}$$

Filter

- Supondo uma lista $[e_1, \dots, e_k]$ de tipo $[t]$, a propriedade p que é aplicada aos elementos da lista é do tipo:

$t \rightarrow \text{Bool}$



Filter

```
1 isEven :: Int -> Bool
2 isEven n = (n `mod` 2 == 0)
```

```
Main> filter isEven [1..6]
[2,4,6]
```

Filter

```
1 nonEmpty :: String -> Bool
2 nonEmpty st = (st /= "")
```

```
Main> filter nonEmpty ["ABC","", "Esporte Clube"]
["ABC", "Esporte Clube"]
```

Filter

- Definição:

```
1 filter :: (t -> Bool) -> [t] -> [t]
2 filter p [] = []
3 filter p (a:x)
4   | p a      = a : filter p x
5   | otherwise = filter p x
```

- Alternativa:

```
1 filter :: (t -> Bool) -> [t] -> [t]
2 filter p l = [ a | a <- l, p a ]
```